PA Consistency Machines

A comparative audit and verified reduction experiments

The supplied 209-, 208-, and 196-state constructions

13 November 2026

Result status. No verified machine with fewer than 196 working states was obtained. This report does not claim that 196 is a lower bound. It gives an exact comparison of the supplied designs, two verified 196-state intermediates, a new logical reduction of the source program, and reproducible certificates distinguishing successful simulations from rejected smaller candidates.

Verified machine results. A new rebracketing retains all 3,812 source instructions and passes all 4,959 symbolic contracts. A second variant transfers the h-only acceptance convention into PA196, has 3,811 source instructions, and passes all 4,958 contracts. Independent literal interpreters verify complete blank-tape startup configurations for both.

Mathematical contributions. A syntactic interpretation supplies the PA-internal converse needed for the full consistency equivalence. A new integrated equality-witness gate removes a primitive logical head and 67 source instruction sites, reducing the program from 3,812 to 3,745 sites. Its best verified implementation has 203 working states. Achieving a physical state count below 196 remains an additional obligation.

Proof status. The machine certificates are independently replayable Python calculations justified by the published unbounded tape-engine lemmas. The new logical and PA-internal arguments are mathematical proofs. This work does not claim a new completed or executed Lean formalization.

The accompanying ZIP contains this PDF, its complete TeX sources, literal transition tables, source-site data, proof and verification scripts, source revision locks, licenses, and selected search receipts.

Contents

1 The target and the result obtained **3**

2 Comparison of the supplied constructions **3**

2.1 Scope, model, and exact source identities . . . 3
2.2 The incoming archive census: twenty graph classes . . . 4
2.3 The original one-state U208 reduction . . . 5
2.4 The eighteen additional 208-state layouts . . . 6
2.5 How published PA209 and PA196 differ from the U208 family . . . 6
2.6 Proof strength and the internal equivalence obligation . . . 7
2.7 The contribution and limits of the ConciseIPRA material . . . 7
2.8 What the independent audit establishes . . . 8

3 Verification at the literal-machine boundary **8**

3.1 Source frames and the complete register file . . . 8
3.2 What each finite contract proves . . . 9
3.3 Startup is checked independently . . . 9

4 A structurally different verified 196-state machine **9**

4.1 Construction . . . 9
4.2 The six finite local identifications . . . 10
4.3 The concrete artifact and its guarantees . . . 11

5 A verified intermediate with an h-only acceptance test **11**

5.1 Command semantics and the change in the source program . . . 11
5.2 The necessary reassociation . . . 12
5.3 Finite and uniform verification . . . 12
5.4 Preservation of the consistency property . . . 12

6 The PA-internal converse for the PA196 command calculus **13**

6.1 Conventions and the translation . . . 13
6.2 The finite-comprehension proof template . . . 14
6.3 The gate, including the coincident-label case . . . 14
6.4 A proof compiler and its internal consequence . . . 15
6.5 From proof syntax to the literal machine . . . 16
6.6 What transfers from ConciseIPRA . . . 16

7 A smaller source program by integrating the equality witness **16**

7.1 The modified gate . . . 17
7.2 Proof recipes available without the deleted head . . . 17

7.3 Recovering the deleted equality-witness head . . . 18
7.4 Recovering the old gate . . . 18
7.5 Deriving the new gate in the old calculus . . . 18
7.6 Exact source-program savings . . . 19
7.7 Additional equivalent gate and encoding variants . . . 19
7.8 Exact command identities and completeness . . . 20
7.9 Verified implementations of the integrated gate . . . 22

8 Why the apparent smaller candidates were rejected 23
8.1 A four-step loop caused by incompatible helper reuse . . . 23
8.2 The return scan and transient zeros . . . 23
8.3 The precise scope of the searches . . . 23
8.4 An append-zero initialization failure . . . 24

9 Reproduction and proof boundaries 24
9.1 One command for the positive claims . . . 24
9.2 Rebuilding this PDF . . . 25
9.3 What has and has not been proved here . . . 25

A An exact finite model for the macro grammar problem 25

B Complete transition table of the rebracketed machine 26

1 The target and the result obtained

The requested target is a deterministic, one-tape, two-symbol machine M with fewer than 196 working states, starting on the all-zero tape, for which

$$\mathsf{PA} \vdash \mathrm{Nonhalt}(M) \iff \mathrm{Con}(\mathsf{PA}).$$

The halting control label is additional to the working-state count. The meaning of Con(PA) is the ordinary arithmetized absence of a PA proof of contradiction, with the proof presentation fixed as in the cited development. Changing to another ordinary proof presentation requires the usual PA-provable primitive-recursive proof translations.

The strict state-count target remains unmet in this work. The positive results below have complete, finite artifacts; a table with a nominal count of 195 is expressly rejected because it fails a required simulation and admits a four-step loop on a legitimate source frame. No inference of nonhalting is drawn from merely running a machine for a long time.

Verified table	States	Source sites	Contracts	Startup steps
Published PA196	196	3,812	4,959	58,154
New rebracketing	196	3,812	4,959	58,154
New h-only variant	196	3,811	4,958	57,922
Integrated witness, initial	206	3,745	4,872	59,164
Integrated witness, compressed	203	3,745	4,872	58,154

The source-site numbers count primitive register instructions. The contract numbers additionally distinguish the zero and positive cases of each decrement. The startup numbers count literal binary tape transitions. These three units are different and are not interchanged in state estimates.

The comparison in Section 2 explains why the one-state U208 deletion cannot simply be applied to PA196. The unbounded verification method and the new rebracketed table are given next. The later sections establish the logical results and document the rejected reductions.

2 Comparison of the supplied constructions

2.1 Scope, model, and exact source identities

All state counts in this comparison refer to working states of a deterministic binary Turing machine, with an additional halting label, a tape infinite in both directions, and an initially blank tape. A missing working-state number is not a state: the census counts distinct source labels in the transition table, not the largest label plus one. In particular the historical PA209 table has 209 working labels and halt label 215, whereas PA196 uses the dense working labels $0, \ldots, 195$ and halt label 196.

The comparison fixes the following repository revisions. The links name immutable revisions; references to a repository's moving `main` branch are not used to identify a table.

ProveIt incoming and ConciseIPRA. ProveIt revision `6ce5acc401f0c84be90dc5c38dc1487e391122c6`.

Published PA196. BusyBeaver revision `407cdc356f6b586635e71ac92be394255f4bfea4`.

Historical PA209. BusyBeaver revision `69d98241a721ead20b4f0b17df9062950b40fbfe`.

The complete literal-file SHA-256 digests are:

`data/pa196.tm` `e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`.

`data/pa209.tm` `0610d978675320f0b4b1ef34619dcd3daa5912632d7037e200dfcc70954fea51`.

These are hashes of the supplied files, including their literal row order and formatting. They must be distinguished from the hashes of canonical transition graphs used below. The audit manifest records both kinds of identifier and the complete provenance of every selected table.

2.2 The incoming archive census: twenty graph classes

All eleven ZIP archives in the pinned `docs/incoming` directory were inventoried. Their names do not denote eleven independent reductions. Some contain alternate presentations of the same table, some contain further 208-state tables, and others record negative searches or uncompiled logical proposals. Table 1 separates these roles. Except for `pa_unary_power_cover`, the archive names have the suffix `_2026-10-29.zip`; the shorter stems are printed for readability.

Table 1: The eleven supplied incoming archives.

Archive stem	Audited role
`U208_ConPA`	Primary 208-state presentation in original graph class A.
`U208_ConPA_Conditional_Context`	Primary A and alternative B; the retained member of the folded pair differs from the first presentation.
`U208_ConPA_Entry_Context`	Primary A and alternative B; its primary and alternative literal tables duplicate the corresponding Conditional Context files.
`U208_ConPA_History`	Primary A and alternative B, with a history-based explanation of the same local reduction.
`u208_research`	Primary B and a Plain alternative in A.
`pa208_research`	Eighteen additional 208-state layouts: nine modifications of each original graph class.
`Below_U208_Research`	Boundary projections and guarded, offset, and source searches; no accepted smaller table.
`ConPA_below208_research`	Comparison, two-row searches, source permutations, and a finite-model obstruction to a logical simplification.
`pa_consistency_research`	Source specialization from 3,909 to 3,904 primitives, but an increase from 197 to 200 syntactic subtrees; no smaller compiled table.
`u208_comparison_research`	Comparison and local/source searches; the expressly rejected 207-state candidate is not an accepted consistency machine.
`pa_unary_power_cover`	A mathematical unary-cover proposal and a separate proof stream, without a compiled literal Turing machine.

The first five rows supply nine designated original 208-state presentations. A fresh, independent finite verifier classifies these into exactly two rooted, action-preserving transition-graph classes, denoted A and B. The additional eighteen designated tables in `pa208_research` belong to eighteen more classes. Thus the selected accepted presentations give

$$9 + 18 = 27 \text{ presentations}, \qquad 2 + 18 = 20 \text{ graph classes}.$$

Here "accepted" means the archive's designated final 208-state construction, together with the independent finite local checks described below. It does not reclassify failed exploratory tables as valid machines, or assert a new proof-assistant certification of the original packages.

The classification does more than compare file hashes. Starting at state 0, the verifier numbers working states by breadth-first discovery, following read-zero before read-one. It retains the read symbol, written symbol, head direction, and target, with a separate distinguished halt symbol. Every working state is graph reachable. Equal serializations therefore give an exact rooted action-preserving isomorphism, with the bijection explicitly available from the traversal. Different serializations prove that no such isomorphism exists. The two original canonical SHA-256 identifiers, in this audit's specified serialization, are

A `43d44b9b61daf753b2b833273c6b970c1787b15b7e4e177a30ca4cd886486702`.

B `7bd9c53560bd49e008cdf4653800e0218c5ec94330e24d2a714f35c26971d63b`.

Canonical hashes produced by a different serialization need not agree. The complete file identities and the eighteen other canonical identifiers are retained in `inputs/manifest.json` and `comparison_results.json`.

2.3 The original one-state U208 reduction

The five original 208-state packages all exploit the same pair of historical native states, 172 and 211. Their savings cannot be added. In the Joint/History naming, retain 172, delete 211, redirect both outgoing rows of 163 that previously entered 211 to 172, and replace the read-zero action of 172 by

$$172, 0 \longmapsto (0, L, 217).$$

An equivalent naming keeps 211 and deletes 172, redirecting the appropriate entry from 116. The differing retained label is not a second deletion.

The justification depends on how the state is entered. Complete predecessor histories establish that an old entry to 172 has the two preceding tape bits 10, whereas an old entry to 211 that scans zero has preceding bit 1. The retained state can move left and use that history bit to select the old behavior. The exceptional transitions expand to exact finite blocks of lengths $7, 1, 3, 1$. At each block endpoint the entire tape and head position agree with the old endpoint, with only the state identification applied. All other transitions commute with that identification.

For an aligned finite old execution, let $N(q, b)$ count old steps in state q scanning b. The construction gives

$$T_{208} = T_{209} + 6N(172, 0) + 2N(211, 0), \qquad T_{209} \le T_{208} \le 7T_{209}.$$

These are positive blocks, so the correspondence preserves both halting and nonhalting. The induction starts in the specified initial control state and applies to every common initial binary tape; it is not based on a long blank-tape sample run.

The two original U208 source layouts have the same ordered 3,909 primitive opcodes and operands and the same 5,083 ordinal control-flow edges. Their binary associations and physical dispatchers differ. The original packages record an exact startup of 48,140 steps and 5,084 uniform source-site contracts. This comparison inspected those proof artifacts but did not rerun their complete older symbolic-runtime derivations. The graph census and the extra local helper verifications are independently reproduced here.

2.4 The eighteen additional 208-state layouts

The additional tables reuse a related idea without reducing the final state count. They delete one state q, retain a state r, and add a new helper, named 500 in the native tables. On the exceptional scanned bit b, the retained state moves left to read the tag left by its predecessor. The helper dispatches to existing continuations. Table 2 lists all choices; each is applied once to each original graph class.

Deleted q	Retained r	b	Helper on 0	Helper on 1	Choices
127	124	0	$0R8$	$1R59$, $1R109$, or $1R125$	3
190	160	1	$0R86$	$1R193$	1
178	168	0	$0R154$ or $0R159$	$1R176$ or $1R225$	4
301	88	0	$0R122$	$1R500$	1

Table 2: Nine neutral helper layouts over each of A and B. All labels are historical native labels, not PA196 dense labels.

Each layout has a net change of zero working states: one deletion and one new helper. The independent verifier checks the exact literal patch, the complete incoming-edge census, and the fixed opposite predecessor bits. It then checks the four relevant two-cell executions while treating the exterior tape as opaque. Every macro has positive length at most five, preserves the entire exterior, and ends at the required old endpoint. Ordinary rows satisfy the quotient correspondence. These finite checks establish the local simulation for all allowed surrounding tapes, rather than just sampled register values. They also explain why merely noticing that two rows have similar continuations is insufficient: the predecessor tag and all uses of the chosen helper must be compatible.

2.5 How published PA209 and PA196 differ from the U208 family

The U209 parents used inside the ProveIt U208 archives are not literally the published PA209 machine at the historical BusyBeaver revision. The shared engine and compiler lineage explain many common native labels and rows, but the proof-command bodies, source layouts, and startups differ. For example, History U208 class A and published PA209 have 195 common native working labels and 304 identical transition rows. This is a useful comparison in the inherited naming, not a conjugacy claim.

The principal quantitative comparison is

Construction	Working states	Physical source sites	Blank startup steps
Original U208 classes A/B	208	3,909	48,140
Published PA209	209	3,948	58,210
Published PA196	196	3,812	58,154

The PA209 source-site count is stated by the `sites_count` theorem in its pinned `M6/Program.lean`; the PA196 source-site count is stated in `PA196/Program.lean`. This audit did not rerun those upstream Lean builds. The published PA196 paper describes its physical state partition as 36 engine states, 13 leaf routes, and 147 states reading 159 distinct source-tree nodes.

PA196 obtains its thirteen-state improvement by changing the logical command construction and its compiled layout together. Its universal-quantifier constructor transposes the Cantor-pairing arguments:

$$U(a, B) = 4p(B, a) + 2.$$

It deletes the old primitive equality/transitivity head ρ_7, adds guarded equality vacuity to the gate, and derives the required reverse membership rewrites and neighborhood equality using separation. It also

mirrors the equality witness, exchanges selected antecedents and axiom arguments, reorders the command items, and stores the resulting 201-item AX body as 68 internal nodes represented by 62 physical states through six context-dependent pairs. Two obsolete outer controls are removed. These are described in the pinned PA196 paper and its associated command and source modules.

An exact native-label table comparison removes 75 old working labels, introduces 62 new labels in the 1300 series, and retains 134 working labels. Of the retained rows, 265 are identical and only the following three change:

q	b	old action	new action
101	1	$1R115$	$1R1367$
164	0	$0L311$	$0R77$
190	1	$1L308$	$1R99$.

The net census is $209 - 75 + 62 = 196$. In particular both native 172 and native 211 are already absent from PA196. The central U208 deletion therefore cannot be applied again to remove one of them.

Two neutral U208 helper families, 127/124 and 301/88, survive far enough structurally to motivate new searches. However the old helpers cannot simply be overlaid on PA196. The independently attempted existing-helper substitutions for 127/124 fail uniform source-site contracts, and the obvious self-loop donors for 301/88 belong to the protected register engine. More extensive finite searches are recorded separately with their precise scope and counterexamples. A failed scoped search is not a lower bound on all equivalent machines.

2.6 Proof strength and the internal equivalence obligation

The published PA196 and PA209 developments distinguish external nonhalting from what PA proves about the machine. For PA196 the advertised internal statement is

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M_{196}) \longrightarrow \mathrm{Con}(\mathrm{PA}).$$

The relevant endpoints are the pinned theorem table, the independence module, and the nonhalting module. The last file explicitly separates external soundness from arithmetic unprovability. The old PA209 article makes the same distinction.

Thus the requested PA-provable biconditional has an additional proof obligation. A proof that PA has no proof of either halting or nonhalting, together with an external soundness proof, does not by itself establish the missing implication inside PA. The appropriate additional route is a primitive recursive syntactic translation from an accepted numerical proof to a PA refutation, with PA proving the correctness of that translation. That argument can use finite proof codes and bounded computations; it must not silently invoke a uniform truth predicate for all PA formulas or PA's own semantic soundness. A separate chapter supplies the proposed converse argument. The comparison here does not attribute it to an upstream theorem that states only the displayed implication.

2.7 The contribution and limits of the ConciseIPRA material

The pinned ConciseIPRA project measures complete standalone Python proof checkers by significant tokens, AST nodes, and normalized source bytes. Its mathematical criterion is bi-interpretability with full first-order intuitionistic primitive recursive arithmetic, with whole numerical domains, actual equality, and identity composites. These are substantive requirements, but they are different from a state census for a binary blank-tape machine with the PA-consistency property. The stated bi-interpretability target does not by itself establish the PA-consistency equivalence needed here.

Several design ideas are directly relevant to minimizing proof searchers: selecting an operation from the shape of a premise; deriving unrestricted transport from a one-sided restricted rule; fusing evaluation

with discharge; replacing a general template operation by simpler operations with explicit proof recovery; and using a syntactic tag to select an introduction mode. PA196 already uses a related principle when it removes a primitive equality head but derives its required applications from other heads.

A new use of these ideas would still need two concrete results: a proof translation preserving the requested PA target, and a literal Turing-machine compilation with a verified state count. A smaller rule inventory or shorter Python checker does not supply either result automatically. Consequently this comparison treats the ConciseIPRA proposals as useful design input, without claiming a state-count improvement from them alone.

2.8 What the independent audit establishes

The supplied `compare_designs.py` is a standard-library verifier for all selected transition tables. It checks row uniqueness, binary actions, target closure, working-state and halt domains, graph reachability, canonical graph serialization, exact helper patches, complete predecessor-tag conditions, and the finite whole-tape local simulations for all eighteen new helper layouts. Its checks use explicit failures rather than relying on Python assertions. It passes both normally and under `python -O`, producing byte-identical receipts.

The machine manifest preserves 27 designated 208-state presentations, the published PA209 and PA196 tables, the two native helper baselines, original paths, and hashes. The archive inventory and this chapter keep uncompiled proposals, failed candidates, scoped search results, local simulation certificates, and proof-assistant endpoints distinct. This comparison chapter establishes no new sub-196 state-count result; any such construction must be identified and verified independently of the census above.

3 Verification at the literal-machine boundary

3.1 Source frames and the complete register file

The verifier independently transcribes the symbolic operations of the published `M6/Runtime.lean`. A clean frame has control state 75, a one immediately to the left of the head, and right-hand tape

$$p\,0^{\,n+48-|p|}\,1\;\mathrm{File}(v_0,\ldots,v_{k-1}),\qquad n\in\mathbb{N},$$

where p is a source address, $k \geq 9$, and

$$\mathrm{File}(\vec{v}) = 0\,1^{v_0+1}\,0\,1^{v_1+1}\cdots 0\,1^{v_{k-1}+1}.$$

All remaining tape cells are zero. The logical register numbers $0,\ldots,8$ are stored in physical positions

$$[6,2,0,4,3,7,8,5,1].$$

The symbolic padding is an affine expression in the arbitrary natural n. The register file is one opaque term, with a recorded operation and marked register where appropriate. It is not replaced by a small test vector.

Four unbounded summaries cover marking the selected register, increment, zero decrement, and positive decrement. Their justification is the existing native tape-engine proofs. The verifier checks all 45 literal transition rows on which those summaries depend. In particular the increment summary includes the appended spare register with value one; changing that spare value without adapting the engine proof is not an allowed optimization in these positive certificates.

3.2 What each finite contract proves

For every source address the checker starts from its symbolic clean frame and follows literal transition rows, finite scan summaries, and the four unbounded register summaries. It must reach the prescribed next clean frame, with the exact complete-file effect and a positive number of literal steps. A halt instruction must reach the designated halting label. Decrement has separate contracts under $v_r = 0$ and $v_r > 0$.

There are 1,147 decrement sites in the published source, hence $3,812 + 1,147 = 4,959$ contracts. The check includes arithmetic-loop bodies and control transfers as well as the straight-line formula constructors. Unfolding a symbolic derivation and applying the engine lemmas gives a finite positive simulation for every admissible natural padding and every complete register file. The mathematical conclusion is therefore universal; it is stronger than a collection of bounded numerical trials.

The release verifier uses explicit exceptions at its acceptance gates and works with Python assertions disabled. The underlying source parser is checked against pinned input hashes and the explicit graph identities. This remains a small, reviewable certificate-replay implementation rather than a replacement for a proof assistant's trusted kernel.

3.3 Startup is checked independently

Symbolic contracts begin at clean source frames, whereas the task starts on a blank tape. A separate interpreter represents the entire tape by the set of its nonzero cells and executes one literal transition at a time. It imports neither the symbolic runtime nor its register summaries. At the claimed startup step it compares the *whole* tape, state, and relative head position with the clean source-25 frame. The initial physical file is

$$[0, 0, 0, 0, 0, 0, 2, 0, 0] +\!\!+ [1]^{26}.$$

For both the published and the rebracketed table this occurs at step 58,154, with absolute head position -2 and state 75. For the h-only table it occurs at step 57,922. The zero extension of an address is part of its padding, so the differing printed source-25 path of the h-only table does not hide a tape discrepancy.

Positive step lengths, deterministic execution, and the one halting label transport the source program's halting behavior to the literal blank-tape machine. These transport arguments can be formalized in PA by bounded induction and primitive-recursive trace construction. Section 6 spells out the additional syntactic argument needed for the consistency biconditional.

4 A structurally different verified 196-state machine

4.1 Construction

The AX body is a sequence over the following macro alphabet. The single-letter names are opaque compression tokens: in particular token A copies the register called D in the arithmetic program, and token D copies the register called A. The mapping, not an intuitive interpretation of the letters, defines the construction.

Token	Original source node	Token	Original source node
P	`node49`	H	`node58`
A	`node55`	D	`node70`
C	`node65`	R	`node72`
S	`node75`	G	`node142`
K	`node150`	i	`inc 1`
B	`breakTo 1`		

The search also exposes the closed copy suffixes h = `node57`, a = `node54`, and d = `node69`, with H = (P, h), A = (P, a), and D = (P, d). Reassociations are accepted only after reconstructing every primitive instruction's ordinal destinations and checking equality with the original 3,812-instruction control graph.

The selected grammar is `verification/open_physical_61_201240.json`. The "61" in this historical search filename is an *obsolete optimistic search score*; it is not the certified state saving. Its first fold proposal contained an incompatible pair. The accepted certificate uses the compatible six pairs below and has 196 states. This potentially confusing filename is retained only so that the research trail remains reproducible.

Compiling its distinct new readers gives 202 working states, comprising the unchanged 134-state exterior and 68 new pure readers. Six compatible predecessor-sensitive identifications reduce 202 to 196. Finally a bijection maps the surviving new reader labels to $134, \ldots, 195$, preserving all labels $0, \ldots, 133$ and the halt label 196.

4.2 The six finite local identifications

In the temporary pure-reader numbering, the entire fold certificate is

Kept	Deleted	Helper for read 0	Helper for read 1
1004	1013	1039	–
1006	1028	1035	1023
1020	1003	1018	–
1029	1009	–	1057
1044	1024	1043	–
1048	1049	1008	–

Every original entry to the kept state has predecessor bit zero, and every original entry to the deleted state has predecessor bit one. A dash means that the two original rows already agree. On a differing read bit b, the shared state preserves b, moves left, and enters the stated helper. The helper reads the predecessor bit, returns right, and selects a donor that performs the original transition.

All 24 local cases are checked *after all six identifications have been applied*. Each takes one or three literal transitions, touches only the two specified cells, preserves their contents, and finishes at the identified old endpoint one cell to the right. This simultaneous check matters: an individually suitable helper may cease to work when another selected fold changes it. Ordinary unchanged rows commute with the identification.

The relabeling verifier checks all 392 rows, including halt targets. All 196 working states are graph reachable. The resulting canonical graph is structurally different from the published PA196 graph, so this is more than a relabeling of the published table.

4.3 The concrete artifact and its guarantees

The complete dense table is `comparison/audit/rebracketed196_dense.tm`; it is also printed in Appendix B. Its SHA-256 is

`e212e359837cff6d1b3ab665855b0e9b4a2b471d23e77db447680d494d943b80`.

It has start state 0, working states $0, \ldots, 195$, and halt label 196. The longest source path decreases from 38 to 37. Its 4,959 contracts have a maximum symbolic execution length of 515 and a total of 1,589,112 symbolic steps, compared with 1,619,679 for the published table. These symbolic step counts are verifier costs, not universal bounds on literal tape runtimes. The startup remains exactly 58,154 literal steps.

Since the primitive source program is unchanged, the all-natural contracts and startup prove halting equivalence with PA196. Together with the mathematical internal converse below, they give

$$\mathsf{PA} \vdash \mathrm{Nonhalt}(M_{\mathrm{rebracketed}}) \leftrightarrow \mathrm{Con}(\mathsf{PA}).$$

The state count is still 196; the construction supplies a verified different starting point for further reductions.

5 A verified intermediate with an h-only acceptance test

This section records a useful intermediate construction. Its table still has 196 working states; it is not, by itself, a solution of the strict state-count problem. It changes only five rows of the published PA196 table and passes 4,958 uniform source-site contracts for a 3,811-instruction source program. The construction transfers the $h = 1$ acceptance convention already present in the U208 family into the newer PA196 arithmetic and command language.

5.1 Command semantics and the change in the source program

Write a command store as (t, w, d, h). A nonhalting Shift changes it to

$$(h, \mathrm{tail}(w), d, \mathrm{head}(w)),$$

where head and tail are the two Cantor-decoding components, including the usual zero-underflow convention. The published PA196 Shift halts if $t + h = 1$. The variant halts if $h = 1$. All other commands and all normal Shift effects are unchanged.

The low-level Shift preparation assumes that its work counter X is zero. Its original three-instruction prefix transfers T into X. Replace that prefix by the two-instruction loop clearing T. The rest of the preparation copies H into T and X, transfers X back to H, increments X, and returns to the common comparison routine. Consequently its register effect before the comparison is

	T	H	X
original	h	$t + h$	1
variant	h	h	1.

The comparison halts exactly when $H = X$. On its normal exit it clears the comparison counters and proceeds to the same stack pop. This proves the claimed command semantics by the existing elementary transfer and comparison lemmas; no claim about arithmetic consistency enters this calculation.

In original source numbering the transfer loop occupies sites 3800–3802. The variant uses site 3800, $\mathrm{dec}\,T$, followed by site 3801, continueTo(1). The $\mathrm{inc}\,X$ is removed and the loop-back depth changes from two to one. All old sites after 3802 have new index one smaller; all sites through 3799 keep their ordinal index. The resulting ordinal control graph agrees with this explicit edit.

5.2 The necessary reassociation

A naive call to the existing clear-T decoder does not work: that decoder shares a physical state with clear-H and relies on the preceding address bit. Merely substituting the source subtree therefore fails two source-site contracts. A first reassociation fixes clear-T but breaks the preceding `continueTo(3)`. The following wider reassociation fixes both contexts.

Let $A =$ `node36`, $B =$ `node167`, $C =$ `node145` (clear T), and $D =$ `node57`. Replace the old subtree

$$(A, (B, (\texttt{node37}, D))) \quad \text{by} \quad ((A, (B, C)), D).$$

Here `node37` is the old T-to-X transfer. No control-transfer operand elsewhere in the tree changes. With the published dense table labels, the complete row patch is

state	read	write	move	next
37	0	0	R	11
37	1	1	R	101
127	0	0	R	133
127	1	1	R	20
133	0	0	R	120.

Every unlisted transition is exactly the PA196 transition. The original halting state remains 196, the initial state remains 0, and the physical source-root state remains 75. All 196 working states remain graph reachable from state 0. The maximum source address length grows from 38 to 39, still within the unchanged 48-cell header.

5.3 Finite and uniform verification

The file `h_catalan_node170_3_133_37.tm` contains the complete table; the adjacent `_sites.json` file gives the ordered source sites. The symbolic runtime check covers every increment, every jump, every halt, and both zero and positive branches of every decrement. It verifies 4,958 contracts. The check treats the padding length as an arbitrary natural parameter and the entire register file as an opaque symbolic block. It therefore does not bound the input values, the register sizes, or the header padding. The register-block summaries use the same protected engine rows as PA196; those rows have not changed.

An independent sparse-tape interpreter, using only literal binary steps, finds the exact canonical initial source frame after 57,922 steps from the blank tape. It checks the complete set of nonzero tape cells, not just a finite window. The head is at absolute position -2, and the current state is 75. Its control address is 1000000, whose extra final zero is absorbed by the zero padding. The physical register file is

$$[0, 0, 0, 0, 0, 0, 2, 0, 0] +\!\!+ [1]^{26}.$$

The same interpreter independently reproduces the published PA196 startup length of 58,154 steps. These receipts are in `h_symbolic_result.json` and `h_startup_result.json`. They are reproducible proof calculations using the published runtime lemmas, not a claim that a newly generated Lean proof has been kernel checked.

5.4 Preservation of the consistency property

The PA196 proof invariant is syntactic: t, h, and every stack member have finite derivations in its numerical calculus. It is not a truth predicate. The invariant is preserved by the new normal Shift for precisely the same reason as by the old one. A new halt has $h = 1$, hence supplies a finite derivation

of the distinguished contradiction code. The syntactic proof translation into PA described separately converts this into a PA refutation. Every operation on a bounded execution and its attached derivations is primitive recursive, and its correctness can be proved in PA. This yields

$$\mathrm{PA} \vdash \mathrm{Halt}(M_h) \longrightarrow \neg\,\mathrm{Con}(\mathrm{PA}).$$

For the other direction, start with the existing compiler from a PA refutation to an old halting command word. Take the execution prefix *before* its first halting Shift. Replay that prefix with the new semantics. If a new Shift accepts sooner, the desired halt is already obtained. Otherwise the two stores agree throughout the prefix. At its end, $t + h = 1$, so exactly one of the following cases occurs.

1. If $h = 1$, append one Shift, which accepts under the new convention.
2. If $t = 1$ and $h = 0$, discard the old final Shift and append $\mathrm{AX}(0, 0, 0)$ followed by ten Shifts.

In the second case the AX command has false gate, sets $t = h = 0$, and pushes its eight generated heads followed by the saved old top 1. All eight heads differ from 1: seven have outer implication tag 0 and the remaining head has universal tag 2. After nine normal Shifts the new h equals the saved 1; the tenth Shift accepts. The older stack contents do not matter. Applying the AX command *after* replaying the old final Shift would be incorrect because that Shift can discard the saved $t = 1$; the construction explicitly stops before it.

This padding operation is primitive recursive in the finite old execution. Together with exhaustive command-word enumeration and the positive finite runtime simulation, it proves within PA

$$\neg\,\mathrm{Con}(\mathrm{PA}) \longrightarrow \mathrm{Halt}(M_h).$$

Thus, at the ordinary mathematical proof level, the intermediate has the same full PA-provable consistency equivalence as the target construction. The statement uses the separately supplied syntactic converse argument; it must not be attributed to the unmodified upstream Lean theorem, which explicitly states only its existing direction.

6 The PA-internal converse for the PA196 command calculus

The published endpoint of PA196 is

$$\mathrm{PA} \vdash \neg H_{196} \longrightarrow \mathrm{Con}(\mathrm{PA}).$$

Its separate external nonhalting theorem uses a concrete model of hereditarily finite lists. That external theorem, by itself, does not establish the opposite implication inside PA. The argument below supplies the opposite implication by a syntactic interpretation. It is an ordinary mathematical proof and a specification for an additional formalization; it is not a claim that the existing Lean declarations already contain this converse.

6.1 Conventions and the translation

Write $\mathcal{C}$ for the numeric calculus of `PA196/Logic/Basic.lean`. Its rules are modus ponens and theorem generalization; its axioms are the six displayed heads, the restricted propositional head, and the guarded gate. A derivation is a finite tree whose nodes record the rule and its numerical arguments. Let $\mathrm{Prf}_{\mathcal{C}}(p, n)$ be the primitive recursive predicate saying that p is such a tree with conclusion code n.

The translation $A \mapsto A^*$ decodes numeric formulas using the actual transposed universal constructor. It replaces membership by binary-bit membership, equality by arithmetic equality, implication by implication, and a quantifier with label a by $\forall x_a$. Precisely,

$$\mathsf{M}(a,b)^* = \mathrm{Bit}(x_a, x_b), \qquad \mathsf{E}(a,b)^* = (x_a = x_b), \qquad \mathsf{U}(a,B)^* = \forall x_a\, B^*.$$

Here $\mathrm{Bit}(s,i)$ means that the binary digit of s at position i is one. Code zero is decoded as $\top$, for example $0 = 0$. The numeric identity $\mathsf{I}(0,0) = 0$ is the sole degenerate implication case: the translation proves $0^* \leftrightarrow (0^* \to 0^*)$, rather than pretending that their parse trees are identical. For every other implication code, decoding gives the indicated implication literally. Consequently the translation of an MP step is either ordinary MP, or the fixed proof of $\top$ in this exceptional case. Code one is $\mathsf{M}(0,0)$ and hence translates to $\mathrm{Bit}(x_0, x_0)$.

We may carry out the proof in the conservative definitional extension $\mathrm{PA}^{\mathrm{pr}}$ with symbols for the fixed primitive recursive functions needed below, including exponentiation and binary digits. Their totality is provable in PA, and the usual elimination of those definitions is a primitive recursive proof transformation. No uniform evaluation of arbitrary primitive recursive programs is being postulated. Only finitely many fixed numerical functions occur in these definitions.

6.2 The finite-comprehension proof template

For every arithmetic formula $\varphi(i, \vec{z})$, with s, n chosen fresh, PA proves the corresponding instance

$$\forall n\, \exists s < 2^n\, \forall i < n\, \big(\mathrm{Bit}(s,i) \leftrightarrow \varphi(i, \vec{z})\big). \tag{1}$$

This uses full induction, so no restriction is placed on the quantifier complexity of φ. At $n = 0$, choose $s = 0$. In the induction step, choose $s' = s + 2^n$ if $\varphi(n, \vec{z})$, and $s' = s$ otherwise. The bound $s < 2^n$ ensures that adding 2^n changes exactly the new bit; the familiar bit identities and $s' < 2^{n+1}$ complete the step. Classical excluded middle for $\varphi(n, \vec{z})$ is available in PA.

This is a syntactic template, not a uniform truth definition. Given the code of φ, a primitive recursive algorithm writes the displayed induction instance and the fixed proof around it. Fresh variable selection, capture-avoiding substitution, the induction-axiom recognizer, and proof concatenation are primitive recursive. PA proves the algorithm's output is a proof of the corresponding instance by checking these syntactic operations. The induction instance changes with φ; no assertion that PA proves its own soundness occurs.

PA also proves

$$\forall a\, \neg \mathrm{Bit}(a,a), \qquad \mathrm{Bit}(a,i) \longrightarrow i < a. \tag{2}$$

For the second assertion, a set bit implies $2^i \le a$, while $i < 2^i$ is provable by induction. The first follows by setting $i = a$.

6.3 The gate, including the coincident-label case

Fix numerical labels a, b, c with $c \neq a, b$. Put $\Phi = (\mathrm{decode}(c))^*$. Every variable label appearing anywhere in $\mathrm{decode}(c)$ is strictly below c. The reason is that a nonzero constructor's Cantor components are smaller than the constructor code, in either order; induction on the decoder proves the assertion. In particular x_c does not occur in Φ, even bound. This syntactic fact is primitive recursively checkable and PA-provable. It is stronger than the required freshness condition.

First suppose $a \neq b$. Under fixed values $A = x_a$ and $B = x_b$, apply (1) at $n = A + B + 1$ to

$$\theta(i) = \big(\mathrm{Bit}(A,i) \vee i = B\big) \wedge \Phi[i/x_b].$$

The resulting number s satisfies

$$\forall i\ (\mathrm{Bit}(A,i) \to \Phi[i/x_b] \to \mathrm{Bit}(s,i)) \wedge (\mathrm{Bit}(s,i) \to \Phi[i/x_b]), \qquad \Phi \to \mathrm{Bit}(s,B).$$

For $i \geq n$, both the first antecedent and $\mathrm{Bit}(s,i)$ are false, by the bound on s and (2); hence the unbounded universal statement follows from the bounded comprehension.

If $a = b$, the occurrence of x_a in the gate's universally quantified membership antecedent is bound together with x_b. That antecedent is then $\mathrm{Bit}(i,i)$, always false. In this case use $n = B + 1$ and $\theta(i) = (i = B) \wedge \Phi[i/x_b]$ instead. The same required separation statement and distinguished-point implication follow. This case distinction is necessary: silently treating x_a as a fixed parameter when $a = b$ would be a variable-capture error.

These two cases give precisely the translation of

$$\exists x_c(S(a,b,c) \wedge P(a,b,c)),$$

where $P = c \to \mathsf{M}(c,b)$ and

$$S = \mathsf{U}(b, (\mathsf{M}(a,b) \to P) \wedge (\mathsf{M}(c,b) \to c)).$$

Because $c \neq a, b$, the equality- and membership-vacuity clauses Δ and V are ordinary quantifier-vacuity tautologies after translation. If $Q = \mathsf{U}(c, S \to \neg P)$, the displayed witness refutes Q. From Δ, V, and $\neg Q$, propositional logic gives

$$\neg(\Delta \to (V \to Q)),$$

which is the exact PA196 gate. When a guard fails, the command returns code zero, already covered by the proof of $\top$.

After using the fixed PA proof that $\mathrm{Bit}(x_0, x_0)$ is false, all six other heads translate into first-order logical or equality theorems. This replacement matters for heads containing encoded negation or existential quantification: their translations are PA theorems, not pure logical validities with an arbitrary predicate in place of falsity. The restricted propositional head translates into a classical tautology; its numerical restriction is irrelevant to soundness. The proof templates also cover the exceptional numeric implication code by the fixed equivalence noted above.

6.4 A proof compiler and its internal consequence

There is a primitive recursive function T for which PA proves

$$\mathrm{Prf}_{\mathcal{C}}(p,1) \longrightarrow \mathrm{Prf}_{\mathrm{PA}}(T(p), \ulcorner \bot \urcorner). \tag{3}$$

To construct it, traverse the derivation tree. Translate each axiom by the templates above, each MP node by MP (with the exceptional truth case handled explicitly), and each theorem-generalization node by arithmetic generalization. At the root append the fixed proof of $\neg\,\mathrm{Bit}(x_0, x_0)$ from (2), followed by the contradiction rule of the selected PA presentation. Eliminate the finitely many definitional extensions. Recursion on a finite tree and substitution into fixed proof templates are primitive recursive; duplication of previously constructed proofs may enlarge certificates but does not change this fact. PA proves correctness by induction on the tree code, using its ordinary syntactic proof-checking predicates.

Thus PA proves

$$\mathrm{Con}(\mathrm{PA}) \longrightarrow \forall p\ \neg\,\mathrm{Prf}_{\mathcal{C}}(p,1). \tag{4}$$

The existing HF interpretation is unnecessary for this implication.

6.5 From proof syntax to the literal machine

The sound-store invariant needed by the command interpreter is purely syntactic: its live formulas have $\mathcal{C}$ derivations and every element of the stack has one too. A zero tail pops to zero, and zero has a fixed derivation. An AX step attaches the appropriate axiom proofs and the theorem-generalization proof. An MP step either attaches an MP proof or returns zero. A Shift step halts only if $t + h = 1$, which forces one of the two live formulas to be code one. Consequently PA proves, by induction on a coded finite command execution, that a halting execution from the zero store gives a $\mathcal{C}$ refutation. This proof uses the existence of finite proof codes, not their truth in a model.

To transport this assertion to a specific literal table, one must also internalize its startup and instruction contracts. The relevant contracts are finite, positive-length simulations on clean unary register frames. Their variable parts are the fixed tape-engine loops, proved by arithmetic induction; their program-reading parts are finite symbolic traces. All register operations, path encodings, and trace concatenations are primitive recursive, and PA proves their totality. They therefore hold also for nonstandard coded frames and traces in arbitrary models of PA. Determinism and positive length exclude an intermediate halt before a prescribed return frame. More explicitly, if a literal run halts within t steps while none of the first $t + 1$ source instructions halts, the total source-step function and the positive instruction contracts produce a literal run lasting at least $t + 1$ steps from that frame. Determinism contradicts the claimed earlier halt. This is a bounded induction argument available inside PA; it does not invoke the truth of an unbounded nonhalting assertion. The identical argument applies to command boundaries using the positive, total arithmetic macro contracts. This gives, within PA, equivalence of blank-tape halting and source-program halting, and hence the literal implication

$$\mathrm{PA} \vdash H_{196} \longrightarrow \neg\,\mathrm{Con(PA)}.$$

Together with the published reverse implication this proves

$$\mathrm{PA} \vdash \neg H_{196} \iff \mathrm{Con(PA)}.$$

For a newly reduced table the same conclusion follows once its finite symbolic instruction contracts and startup preserve this source program. This mathematical proof should be distinguished from the stronger artifact claim of a completed Lean formalization of the new internal converse: no such completed formalization is supplied by the upstream PA196 endpoint.

6.6 What transfers from ConciseIPRA

ConciseIPRA is useful here as a design principle: replace primitive logical and equality heads by derivations, exploit a deliberately chosen representation, and count the implementation of the resulting checker rather than surface axiom symbols. PA196's removal of the primitive seventh equality head is exactly this kind of reduction. The mathematical target cannot, however, simply be replaced by the ConciseIPRA calculus. Its stated equivalence is with intuitionistic primitive recursive arithmetic, whose induction is limited to equations after the appropriate translation. Our finite-comprehension template deliberately uses induction for arbitrary arithmetic formulas. A machine searching only for an iPRA refutation would establish the consistency of that theory; the required PA-internal equivalence with Con(PA) would require an additional theorem and does not follow from ConciseIPRA's bi-interpretation theorem.

7 A smaller source program by integrating the equality witness

This section gives a new reduction of the proof-search *source program*. It preserves the exact numeric proof calculus up to primitive-recursive proof translations, while removing one independently generated

AX stack item. It does not, by itself, establish a reduction in the number of physical Turing machine states: the final decoder grammar, context folds, return scan, and blank startup must still be certified separately.

7.1 The modified gate

Use the original numeric constructors I, M, E, U, with U_aB denoting $U(a, B)$, $A \to B = I(A, B)$, and $NA = A \to 1$. Thus 1 is the distinguished falsity code, not the truth value true. For parameters a, b, c, put

$$\begin{aligned} &F = E(a,b), && \Delta = F \to U_cF, && V = M(a,b) \to U_cM(a,b),\\ &\Pi = c \to M(c,b), && S = U_b\big((M(a,b) \to \Pi) \wedge (M(c,b) \to c)\big),\\ &X = U_b(NF), && W = NX = \exists b\, E(a,b). \end{aligned}$$

Here c in a formula position means the formula with numeric code c; in a quantifier subscript or atomic variable position it is a variable label. This is the same convention as the original guarded gate.

The 196-state design's unguarded bundle is

$$\begin{aligned} Q &= U_c(S \to (\Pi \to 1)),\\ G &= N(\Delta \to (V \to Q)). \end{aligned}$$

Replace its inner falsity occurrence by X:

$$\begin{aligned} Q' &= U_c(S \to (\Pi \to X)),\\ G' &= N(\Delta \to (V \to Q')). \end{aligned}$$

The runtime guard is unchanged: return numeric 0 when $c = a$ or $c = b$, and otherwise return G'. Delete the separate fifth head $W = \exists b\, E(a, b)$ from the AX output stack. Retain heads 1, 2, 3, 4, and 6, the restricted propositional schema, modus ponens, and generalization.

Semantically, for $c \neq a, b$, the new bundle packages the old bundle together with the equality witness. The following syntactic argument establishes the required equivalence without assuming a truth predicate or a general first-order completeness theorem.

7.2 Proof recipes available without the deleted head

We use finite proof constructions already exhibited in the source repository: `PA196/Logic/Deduction.lean` supplies distribution `r2`, quantifier exchange `r4`, `explosion`, and double-negation elimination; `PA196/Logic/Vacuity.lean` supplies `negImpLeft`, `negImpRight`, `negationVacuity`, `universalVacuity`, and `liftImp`. Propositional composition, exchange, and deduction are constructed from the restricted propositional schema in `ImplicationBasis.lean` and its dependent modules.

The recipes used below do not invoke the fifth head. In particular, `explosion` uses the retained first head; distribution uses the retained sixth head; quantifier exchange uses the retained second head. The negation vacuity construction uses these same retained rules. This dependency check is essential: the fifth head is recovered using these recipes, so using it inside one of them would be circular.

In ordinary formula notation, the two vacuity recipes say

$$A \to U_cA \quad \Longrightarrow \quad NA \to U_c(NA), \tag{5}$$

$$A \to U_cA \quad \Longrightarrow \quad U_bA \to U_c(U_bA). \tag{6}$$

For the second implication, generalize the premise over b, apply quantifier distribution, and then exchange U_bU_c with U_cU_b. The first implication is the existing finite proof of `negationVacuity`, using normality, the first head, contraposition, and double-negation elimination. These are uniform proof constructors, not additional inference rules.

7.3 Recovering the deleted equality-witness head

Fix arbitrary a, b and choose

$$c = \max(a, b) + 1.$$

The new gate is therefore available. Propositional elimination from G' gives proofs of

$$\Delta, \qquad V, \qquad NQ'.$$

Apply (5) to $\Delta = (F \to U_cF)$, then (6) to NF. This yields

$$NF \to U_c(NF), \qquad X = U_b(NF) \to U_cX.$$

The propositional implication

$$X \to (S \to (\Pi \to X))$$

can be generalized over c and lifted by quantifier distribution. Hence $U_cX \to Q'$, and composition gives $X \to Q'$. Together with NQ' this proves NX, which is exactly

$$\exists b\, E(a, b).$$

Thus every instance of the deleted fifth head has a derivation in the new calculus. The choice of c is primitive recursive, and the proof uses no fifth-head instance.

7.4 Recovering the old gate

For each guarded triple, explosion gives $1 \to X$. Ordinary implication monotonicity, first under $\Pi \to _$ and then under $S \to _$, gives

$$S \to (\Pi \to 1) \quad \longrightarrow \quad S \to (\Pi \to X).$$

The `liftImp` recipe for U_c therefore supplies $Q \to Q'$. By contraposition, $NQ' \to NQ$. The three consequences Δ, V, NQ' extracted from G' now provide Δ, V, NQ, and propositional introduction rebuilds G.

This direction does not require the recovered fifth head: it uses only explosion and the retained quantifier-distribution head. Consequently the new calculus proves every old primitive axiom by an explicitly finite construction.

7.5 Deriving the new gate in the old calculus

Conversely, the old fifth head supplies $NX = X \to 1$. Generalization over c provides $U_c(X \to 1)$. Propositional implication monotonicity and quantifier distribution then give

$$Q' \to Q.$$

The old gate yields Δ, V, NQ. Contraposition gives NQ', and these three formulas rebuild G'. Thus the new gate is a theorem of the old calculus; all other retained primitive axioms and both inference rules are identical.

Replacing each primitive-axiom node of a finite proof tree by its indicated finite template yields primitive-recursive proof translations in both directions. The translations preserve the final formula, including the falsity code 1. Their correctness is provable in PA by induction on proof trees. In particular, the old and new calculi have provably equivalent consistency statements. The conditional output 0 at the two excluded guard cases remains the original harmless truth code.

7.6 Exact source-program savings

The postfix macro alphabet is the one used in the original AX grammar. Its *macro* P saves the current top and starts a zero expression; i increments that expression; C combines two saved expressions. These macro letters must not be confused with the logical point formula Π above. The authoritative token-to-source-node mapping is included with the construction scripts.

The original fifth-head word is

$$\mathsf{HACiiiPiCACiiPiC}.$$

Its final PiC performs the outer negation. The remaining prefix

$$\mathsf{HACiiiPiCACii}$$

constructs X. Remove the whole 16-token fifth-head word and replace the inner falsity-building word Pi in the old gate by this 13-token prefix. Therefore the AX macro word changes by

$$201 - 16 - 2 + 13 = 196$$

tokens. The net deletion is exactly $2\mathsf{P} + 2\mathsf{i} + \mathsf{C}$. In the supplied source tree, P expands to 19 primitive instruction sites and C to 27. Hence the complete source program changes by

$$3812 - (2 \cdot 19 + 2 + 27) = 3745$$

primitive instruction sites. A symbolic postfix-expression evaluator checks that the resulting formula is exactly G'; these counts are computed from the actual expanded trees, not estimated from displayed notation.

The AX command now pushes seven generated formulas followed by the saved incoming top, for eight stack items in total. The old nine-Shift identity becomes an eight-Shift identity, and the old ten-Shift stack-pop construction uses nine Shifts. The surviving head selectors are adjusted to their new positions. The deleted head is selected by the finite gate proof above. All intermediate generated heads still have implication or universal outer tags, so the same elementary nonacceptance checks used by the original stack identities apply. These changes give the corresponding primitive-recursive finite command-word translations and preserve the intended consistency search.

Physical certification remains separate. The integers 196 and 3745 in the preceding computation count AX macro tokens and source instruction sites, respectively. Neither is a physical state count. Any literal Turing machine built from this source requires its own complete transition table, source-site simulation certificates, and blank startup check. A source-level reduction must not be reported as a sub-196-state construction unless those additional checks succeed.

7.7 Additional equivalent gate and encoding variants

Two independent curried-implication exchanges give four gate words: exchange Δ, V in the outer chain, and exchange S, Π in the inner chain. The propositional exchange theorem, lifted under U_c where necessary, provides derivations in both directions. All four words have 196 opaque macro tokens.

A second witness orientation is also available. Set

$$X^{\mathrm{mir}} = U_a(NE(b,a))$$

instead of X. The resulting witness family is the same fifth-head family, with its two parameters exchanged. Its vacuity proof uses the *other* new gate, at (b,a,c), to obtain $E(b,a) \to U_cE(b,a)$. Negation and universal vacuity then give $X^{\mathrm{mir}} \to U_cX^{\mathrm{mir}}$. The gate at (a,b,c) supplies the contradiction needed to infer NX^{mir}. Both guard conditions hold when $c = \max(a,b)+1$. Thus this orientation requires two gate instances but neither equality symmetry nor the deleted fifth head. Combining it with the two exchanges gives eight equivalent 196-token gate words.

For completeness, the search also considered independently reversing the numeric Cantor-argument order of M, E, U. A precise coding transport avoids a subtlety here: a gate parameter is simultaneously a formula code and a variable label. Transport both roles by the same map τ. With $\langle\cdot,\cdot\rangle$ the original Cantor pairing, define $\tau(0) = 0$. For $n > 0$ uniquely written as $n = 4\langle u, v\rangle + k$, transform its two arguments recursively and interchange them exactly when tag k is one of the chosen tags from $\{1,2,3\}$. Tag zero, the implication constructor, is never interchanged. Equivalently,

$$\tau(4\langle u,v\rangle + k) = \begin{cases} 4\langle \tau(v), \tau(u)\rangle + k, & k \text{ is reversed,} \\ 4\langle \tau(u), \tau(v)\rangle + k, & k \text{ is retained.} \end{cases}$$

Every proper constructor argument is less than its code, so this is a primitive-recursive course-of-values definition. Induction on codes proves $\tau(\tau(n)) = n$; in particular it is a bijection, fixes both 0 and 1, and preserves the gate inequalities $c \neq a, b$ when all three parameters are transported. The dual role of c is preserved exactly, since both its variable label and its formula code become $\tau(c)$.

One must not assume that τ preserves numerical order. The side condition $u \leq a$ on a primitive restricted propositional axiom may fail after transport. Instead, translate such an axiom using the already-derived full propositional basis. Those finite proof constructions rely on the unchanged implication constructor's numerical bounds and on the retained first head, so they remain available in the recoded calculus. All other primitive axioms transport directly, as do modus ponens and generalization. This supplies primitive-recursive proof translations preserving falsity in both directions. It justifies the 64 combinations of gate and constructor orientations investigated by the supplied search scripts, without asserting that any of those encodings reduces the physical state count.

7.8 Exact command identities and completeness

Here are the command-level details behind the shorter-stack statement. Write stores as $\langle t, w, d, h\rangle$, let $\langle a, w\rangle_C$ denote Cantor stack cons, and let Sh be the unchanged Shift command: it halts exactly when $t + h = 1$, otherwise replacing the store by $\langle h, \mathrm{tail}(w), d, \mathrm{head}(w)\rangle$. Modus ponens is also unchanged. On input $\mathrm{MP}(x)$, set $y = d + x$; if $I(t,y) = h$, return $\langle y, \langle 0, w\rangle_C, y, 0\rangle$, and otherwise return $\langle 0, w, y, 0\rangle$.

Let $D = d+x$, $H = h+y$, and $A = c$. The modified command $\mathrm{AX}'(c,x,y)$ returns the guarded $G'(H,D,c)$ as its current top, clears d, h, and pushes, in top-to-bottom order,

$$q_1, q_2, q_3, q_4, q_5, q_6, q_7, t,$$

where

$$(q_1,q_2,q_3,q_4,q_5) = (\mathrm{head}_6(D,A,H), \mathrm{head}_4(D,A,H), \mathrm{head}_3(D,A,H), \mathrm{head}_2(D,A,H), \mathrm{head}_1(D,H)),$$
$$q_6 = \rho_8(D,A,H,d), \qquad q_7 = U_dh.$$

The displayed five-tuple is schematic notation for the indicated five entries, not an additional numeric encoding.

The guarded top and $q_1, \dots, q_6$ are all divisible by four; q_7 is congruent to two modulo four and is at least two. Consequently the following identities are total for *arbitrary* incoming t, w, d, h, c, x, y:

$$\mathrm{AX}'(c,x,y);\mathrm{Sh}^7 : \langle t,w,d,h\rangle \longmapsto \langle \rho_8(d+x,c,h+y,d), \langle t,w\rangle_C, 0, U_d h\rangle,$$
$$\mathrm{AX}'(c,x,y);\mathrm{Sh}^8 : \langle t,w,d,h\rangle \longmapsto \langle U_d h, w, 0, t\rangle.$$

No assumption that the incoming caches vanish is used. During the first eight Shifts, the tested sums either are divisible by four or contain a summand congruent to two with the other summand divisible by four. The next Shift, if executed, is also safe because its top is $U_d h \geq 2$.

Define the following finite command words:

$$\begin{aligned}\mathrm{Push}' &= \mathrm{AX}'(0,0,0);\mathrm{Sh}^7;\mathrm{MP}(0),\\ \mathrm{Pop}' &= \mathrm{AX}'(0,0,0);\mathrm{Sh}^9,\\ \mathrm{Discard}' &= \mathrm{MP}(1);\mathrm{AX}'(1,0,0);\mathrm{Sh}^8;\mathrm{MP}(0).\end{aligned}$$

They satisfy

$$\begin{aligned}\mathrm{Push}' &: \langle t,w,d,h\rangle \longmapsto \langle 0, \langle t,w\rangle_C, 0, 0\rangle,\\ \mathrm{Pop}' &: \langle t, \langle a,w\rangle_C, d, h\rangle \longmapsto \langle t, w, 0, a\rangle,\\ \mathrm{Discard}' &: \langle t,w,0,0\rangle \longmapsto \langle 0,w,0,0\rangle.\end{aligned}$$

For Push, the final MP comparison fails because its proposed implication code is divisible by four whereas the cache contains $U_d h$, congruent to two. Pop follows from the eight-Shift identity and one additional safe Shift. For Discard, the initial MP fails since $I(t,1) > 0$; it sets $d = 1$. The AX guard then returns zero because its third parameter equals its second variable label. The final MP again fails and clears the caches.

Let $\mathrm{Offset}(u)$ be empty when $u = 0$, and otherwise the single command $\mathrm{MP}(u)$. From $\langle 0,w,0,0\rangle$ it returns $\langle 0,w,u,0\rangle$. For generalization use

$$\mathrm{Gen}'(i) = \mathrm{Push}';\mathrm{Offset}(i);\mathrm{Sh};\mathrm{AX}'(i,0,0);\mathrm{Sh}^8\,.$$

It returns $\langle U_i a, w, 0, 0\rangle$ from $\langle a,w,0,0\rangle$. This uses only the stack identities, not the content of the deleted head.

For modus ponens use

$$\mathrm{MPSuffix}'(b) = \mathrm{Sh};\mathrm{Sh};\mathrm{MP}(b);\mathrm{AX}'(b,0,0);\mathrm{Sh}^9\,.$$

Starting from $\langle 0, \langle a, \langle I(a,b), w\rangle_C\rangle_C, 0, 0\rangle$, it halts on its second Shift if $a = 1$; otherwise it returns $\langle b,w,0,0\rangle$. The first two Shifts expose a and $I(a,b)$, MP succeeds, and Pop removes the resulting saved zero.

For $1 \leq j \leq 5$, a selector for q_j is

$$\mathrm{Select}'(j,D,H,A) = \mathrm{Discard}';\mathrm{AX}'(A,D,H);\mathrm{Sh}^{j+1};(\mathrm{Pop}')^{7-j}.$$

It preserves the incoming stack and returns q_j with both caches zero. The final popped item is the saved zero introduced after Discard. The surviving paper heads have positions $6 \mapsto 1$, $4 \mapsto 2$, $3 \mapsto 3$, $2 \mapsto 4$, $1 \mapsto 5$. The restricted propositional selector, for $u \leq a$, is

$$\mathrm{Discard}';\mathrm{Offset}(u);\mathrm{AX}'(b,a-u,c);\mathrm{Sh}^7;\mathrm{Pop}',$$

which returns $\rho_8(a,b,c,u)$. A selector for the guarded new gate is

$$\mathrm{Discard}';\mathrm{AX}'(c,b,a);(\mathrm{Pop}')^8.$$

These selectors supply every primitive axiom of the modified calculus. The fifth head is first expanded by the finite gate proof already given, then compiled as a derived proof.

Induction on a modified-calculus derivation now gives the same operational compiler invariant as the original construction: from any $\langle t, w, 0, 0\rangle$, its compiled word either halts early or returns the proved formula as top, preserves w, and clears both caches. The axiom cases use the selectors above; the generalization case uses Gen$'$; the MP case compiles the implication, pushes it, compiles the antecedent, pushes it, and executes MPSuffix$'$. An early halt in the antecedent-1 case is the required successful refutation detection.

For a derivation of 1, prepend AX$'(0,0,0)$ and append one Shift. The leading AX clears arbitrary incoming caches, so this finite word halts from *every* incoming store. The unchanged outer scheduler's universal-word coverage argument therefore applies. Conversely, the usual store invariant that every live formula is derivable is preserved: the new gate is a primitive axiom, the remaining generated heads are retained axioms, the $U_d h$ entry is justified by generalization, and MP and Shift have their original meaning. Thus acceptance still yields a derivation of 1.

All of these word constructions, finite proof translations, and execution identities are primitive recursive and admit the same PA-internal finite-run formalization as the original command compiler. This completes the command-level consistency equivalence independently of the eventual physical state allocation. It is a mathematical proof specification; the shortened word library has not been claimed to be a newly completed Lean formalization.

7.9 Verified implementations of the integrated gate

The new logical source has a complete literal implementation in `comparison/audit/integrated_gate206_dense.tm`. It uses working states $0, \ldots, 205$, start state 0, and halt label 206. State 196 is now an ordinary working state and must not be interpreted as halt when executing this table. Its SHA-256 is

`7be1c776f1c5ac1db48b082b1880c710e0ce5722ea82fd66bc33e4ac561afe73`.

The release verifier reconstructs the exact macro-word edit and checks its postfix expression against the displayed gate G'. It reconstructs all 3,745 expanded primitive sites from the modified source grammar. All 1,127 decrement sites have both branches checked, giving $3,745 + 1,127 = 4,872$ uniform contracts. The largest address has length 38; the longest symbolic trace uses 524 verifier steps. The unchanged 45 engine rows retain their existing unbounded summaries. This particular grammar uses two extra return scanner states.

An independent literal blank-tape execution reaches the exact initial source frame after 59,164 steps, at head position -2 and state 75. All 412 rows commute with the documented relabeling, and all 206 working states are graph reachable. The raw and dense tables have identical startup configurations under that relabeling.

A further reassociation and compatible-fold search gives a 203-state table in `comparison/audit/integrated_gate203_dense.tm`, with start state 0, working states $0, \ldots, 202$, and halt label 203. Its SHA-256 is

`c2da8e535f77ba8441ca443ad6f7e93262096eb8dc99373152a64d2774fb5e21`.

The release verifier reconstructs its grammar, proves equality of the expanded 3,745-instruction ordinal control graph with the initial integrated source, and checks all 4,872 contracts. The largest source path is 39; the longest symbolic trace uses 515 verifier steps. This table requires no extra return-scanner states. All 406 rows pass the complete relabeling check and all 203 working states are graph reachable. Its exact blank startup is 58,154 steps, with the same whole initial source frame, head -2, and root 75 as PA196.

Thus the source reduction has certified machine realizations. The best one currently costs seven more physical states than the published PA196 table, illustrating why source length and decoder-state count must be measured separately.

8 Why the apparent smaller candidates were rejected

8.1 A four-step loop caused by incompatible helper reuse

The retained diagnostic `verification/candidates/warm_physical_61_8047.tm` has 195 working labels. It was produced by an optimistic combination of reader folds and a longer return scan. It is not an accepted construction.

The release verifier starts it at the valid clean frame for source site 1111, with nine zero-valued live registers and 26 one-valued spare registers. After 26 literal transitions the entire finite tape, head, and control enter a cycle. The four control states are

$$1024 \longrightarrow 1056 \longrightarrow 1038 \longrightarrow 1018 \longrightarrow 1024.$$

The complete configuration repeats at step 30. Thus the claimed finite source return does not occur. This is an exact finite-tape counterexample to the proposed simulation, independently replayed without symbolic register summaries. It is not merely a timeout at an arbitrarily chosen fuel limit.

The counterexample refutes the candidate's claimed compiler certificate. It is not presented as a general decision procedure for consistency-machine correctness, nor as a proof that every 195-state table fails the task.

8.2 The return scan and transient zeros

Another obstruction occurs even before incompatible folds are applied. The unchanged return routine distinguishes the left tape margin using a long run of zeros. Rebracketing changes the runs of zeros in source addresses. Decrement preparation can introduce an additional transient zero, so a bound on the unmodified address alone is insufficient.

For example, two optimistic grammar candidates with seven-zero address runs pass most of their source contracts after adding one scan state, but fail 14 and 18 decrement contracts respectively. Adding a second scan state makes their pure-reader versions pass all 4,959 contracts. Their compatible reader folds then leave 197 working states. The apparent saving vanishes when both the scan and simultaneous helper obligations are met.

Likewise, deleting the final state of the original return detector does not establish a 195-state machine: the initial comparison address already supplies a failing case. Source rebracketing, contextual macro entry, and return-scan behavior have to be solved together.

8.3 The precise scope of the searches

The archive retains deterministic search scripts and selected receipts for several families: direct identifications, mixed transition-row choices, local detours, existing-helper substitutions, associativity changes, exposure of closed macro bodies, altered acceptance conventions, physical register allocations, axiom-head permutations, and equivalent logical orientations. A separate integer program explores reusable macro phrases. A solver time limit or an incumbent cost is not an optimality certificate.

Some particularly concrete negative results are:

- The original direct nonengine fold pass tested 33,150 candidates; the mixed-row pass tested 28,440 candidates. Their protected engine and frame conditions are stated in the scripts.
- All 36 single transpositions of the nine physical register positions were verified at the source boundary, and the corresponding direct-fold passes tested 1,193,400 candidates without an accepted deletion.
- All $8! = 40,320$ register allocations fixing the original candidate position were scored against compatible predecessor folds on the pure 202-state grammar. Each retained the same best six-state saving.
- Several searches over hundreds of thousands of grammar mutations returned valid or promising 196-state layouts, but no certified smaller literal table. The results depend on their specified grammar and helper families.

These are scoped computational results. They do not prove that the published machine is locally optimal under every transformation, that a particular source language is minimal, or that a sub-196 consistency machine cannot exist.

8.4 An append-zero initialization failure

One experiment changes only the spare-register append action

$$(125, 0): \quad 1L39 \longmapsto 0L39.$$

Even if later register contracts are reinterpreted to permit zero-valued spares, this edit fails the required blank initialization. After exactly 76 steps the machine is in state 35 at head position 4, and the only nonzero tape positions are -3 and 1. Its row on zero is $1R35$. Every cell at or to the right of the head is therefore zero, and each next step writes behind the new head and advances right in the same state. This proves that the modified machine never reaches the intended initial source frame. The release verifier reproduces the complete configuration and the escaping-loop premise. The rejected edit is not included among the positive machine constructions.

9 Reproduction and proof boundaries

9.1 One command for the positive claims

Extract the ZIP and, from its top-level directory, run

```
python3 verify_release.py
python3 -O verify_release.py
```

The standard library is sufficient for these release checks. The script replays the published and both new 196-state tables, verifies exact source identity or the documented h-only source edit, checks all unbounded-engine rows, verifies the six simultaneous folds and complete relabeling, replays blank startup, reproduces the four-step rejected-candidate witness, and repeats the 208-state graph census and neutral-helper checks. It also reconstructs the shortened integrated-gate source, checks its symbolic formula and all 4,872 source contracts for each realization, verifies the 206- and 203-state relabelings and blank startups, and reproduces the append-zero failure. It writes its detailed outputs under `receipts/`.

The arithmetic behind the unbounded register summaries is in the pinned upstream source snapshot under `sources/BusyBeaver/`. The independent Python transcription is `verification/symbolic.py`. The full comparison inputs and their byte hashes are in `comparison/audit/inputs/`. The grammar optimization scripts are research tools; their output must pass the release-style checks before it is treated as a machine construction.

9.2 Rebuilding this PDF

On an ordinary TeX Live installation,

```
pdflatex report.tex
pdflatex report.tex
```

rebuilds the PDF. The document uses Latin Modern, standard mathematical packages, `booktabs`, `longtable`, `xurl`, and `hyperref`. The included build script also documents the local-format fallback used in the execution environment. All TeX inputs are in the ZIP.

9.3 What has and has not been proved here

The exact finite table comparisons, graph identifications, literal startups, source-site replay calculations, and failed-cycle witness are executable results. The mathematical interpretation and integrated-gate proof provide explicit uniform constructions and explain their internalization in PA. They are separate from a completed kernel-checked formalization. The upstream Lean declarations retain their original claims and identities; no new formal theorem is attributed to them.

The final state-count conclusion is limited and explicit: the supplied archive contains verified new 196-state variants and a smaller logical source program with a verified 203-state implementation. It does not supply the requested verified machine with fewer than 196 working states.

A An exact finite model for the macro grammar problem

This experiment optimizes a binary straight-line grammar for the fixed macro word. It does not by itself optimize the complete Turing machine: physical state folds, context-sensitive macro entries, the return scanner, and blank initialization remain separate obligations.

Let w be a word of length n over the opaque macro alphabet. Let $\mathcal{R}$ contain every distinct substring of w of length at least two which occurs at least twice, together with w itself. For the original 201-token AX word, there are 243 repeated substrings, of maximum length 13. Introduce a binary variable x_s for each $s \in \mathcal{R}$, and require $x_w = 1$.

For each $s = s_0 \cdots s_{m-1}$, construct a directed acyclic graph on vertices $0, \ldots, m$. An arc $i \to j$ is available when $s[i:j]$ is a single terminal or is a strictly shorter member of $\mathcal{R}$. Its binary variable is $y_{s,i,j}$. Impose a flow of value x_s from 0 to m. If the arc uses a nonterminal $t = s[i:j]$, also impose $y_{s,i,j} \le x_t$. Thus every selected phrase chooses exactly one partition into terminals and selected shorter phrases. The objective is

$$\min \left(\sum_{s,i,j} y_{s,i,j} - \sum_{s} x_s \right).$$

A selected phrase partitioned into k pieces costs $k - 1$, the number of binary joins needed to assemble those pieces.

Why this model is exact for the abstract grammar count. Any feasible solution is an acyclic n-ary straight-line grammar. Binarizing its rules uses at most the displayed objective number of binary rules; accidental duplicate subtrees can only reduce that count. Conversely, in a binary straight-line grammar, duplicate nonterminals with the same expanded word may be identified. Any nonterminal whose expanded word occurs only once in w can be substituted at its unique use, making a longer n-ary rule. This substitution preserves the sum of $k - 1$ over all rules. Repeating it leaves only the root and

repeated substrings, and therefore gives a feasible flow solution with no greater cost. The two optimum values consequently coincide.

The physical compiler already contains the reader for the phrase ii. A second model credits this fixed reader by decreasing the coefficient of x_{ii} from -1 to -2. Its costs count additional reader states rather than all abstract binary rules. This credit does not include any contextual state folds.

Binarization and path constraints. The supplied binarizer performs interval dynamic programming on the pieces of each selected n-ary rule. It tracks the number of leading zero edges, the maximum expanded path length, and the required incoming edge for the context-sensitive macros R, S, G. It rejects a realization with a zero run longer than the requested bound. A subsequent experiment allowed seven-zero paths and charged two additional return-scanner states. The count is still only a candidate score until all symbolic source contracts and the blank initialization have been checked.

There is also an optional, stronger MILP restriction which assigns integer leading-zero bounds to phrases and requires every nonfinal piece to fit under a left edge. This guarantees a bounded right-comb realization when macro contexts are ignored. It is a sufficient restriction for that chosen family of realizations, not a complete characterization of every possible binary bracketing.

Observed results and their limits. The script `exact_grammar_milp.py` uses SciPy's interface to HiGHS. The original-word raw-count run reached 68 rules and reported a lower bound of 62 after a 180-second limit. The run crediting the existing ii reader reached a paid cost of 67 with reported lower bound 62 at the same limit. Neither is an optimality certificate. A separate feasibility run with paid cost at most 66 reached its 180-second limit without finding an incumbent. The cost-at-most-67 run with the extra leading-zero restriction also timed out without an incumbent. Neither timeout proves infeasibility.

For the first four 196-token integrated-gate words, the corresponding paid cost incumbents were 69, 70, 71, and 71 after 180 seconds, with reported lower bounds 64, 64, 64, and 65. Of four mirrored-witness variants, one reached cost 72 with lower bound 65 after 120 seconds; the other three found no incumbent under the imposed upper bound of 72. The retained logs and JSON grammars record these solver outcomes. These experiments did not produce a verified machine with fewer than 196 states.

B Complete transition table of the rebracketed machine

The start state is 0, the blank symbol is 0, and H denotes halt label 196. An entry wDq writes w, moves one cell in direction D, and enters q. The three blocks below cover every working state and both scanned symbols.

q	Read0	Read1
0	1R1	1R61
1	1R82	1R1
2	0L2	1L3
3	0L4	1L3
4	0L6	1L3
5	1R7	0L5
6	0L9	1L3
7	0R7	0R10
8	0R14	1R13
9	0L15	1L3
10	0L16	0R10
11	0L26	1R18
12	0R19	1R20
13	0L79	1L31
14	0L11	0R22
15	0L23	1L3
16	1L2	0L2
17	0R24	0R32
18	0R84	0L95
19	0R28	1R29
20	0R30	1R31
21	0R24	0R57
22	0R33	1R22
23	0L34	1L3
24	0L35	0R1
25	0R36	1R25
26	0R122	1R41
27	0R38	0L39
28	0R40	1R41
29	0R42	1R43
30	0R44	1R45
31	0R21	1R46
32	0R47	1R32
33	0R32	1R33
34	0L69	1L3
35	1R35	1L2
36	0R49	1R36
37	0R28	1R20
38	0L11	0R36
39	0L5	1L51
40	0R24	0R25
41	0R13	0L16
42	0R53	1R54
43	0R55	1R56
44	0R24	0R52
45	0R58	1R59
46	0R60	0L16
47	0R25	1R47
48	0L35	0R61
49	0R16	1R49
50	0R63	1R64
51	0L39	1L51
52	0R22	1R52
53	0L11	0R49
54	0R58	0L16
55	0R65	1R66
56	0R59	1R67
57	0R52	1R57
58	0R48	0R25
59	0L28	1L68
60	0L102	1L30
61	0R0	1R61
62	1R125	1L51
63	0R70	1RH
64	0L126	1R31
65	0R40	1R60
66	0L79	1R71
67	0R53	1L72
68	0R73	1R74
69	0R69	1R75
70	0R76	1R77
71	0R79	1L110
72	0L98	1R54
73	0R48	0R33
74	0R81	0L41
75	0R50	1R132
76	0R130	1R21
77	0R101	1R90
78	0R92	1R93
79	0R73	1R72
80	0R91	1R119
81	0R95	1R96
82	0R62	0R1
83	0R27	1R97
84	0R89	1R88
85	1L99	1L85
86	0R19	1R100
87	0R58	1R58
88	0R117	1R115
89	0L26	1R195
90	0R95	0L131
91	0R102	1R103
92	0R67	1R56
93	0R55	1R104
94	0R21	1R105
95	0R21	0L10
96	0R59	1R41
97	0R106	1R8
98	0R91	1R107
99	0L131	0L85
100	0R108	1R94
101	0L109	0L2
102	0R38	1R68
103	0R59	1R110
104	0R59	1R111
105	0R66	0L16
106	0R14	1R110
107	0R21	1R112
108	0R113	1L20
109	0R30	1R65
110	0R38	1L108
111	0R121	0L16
112	0R114	0L16
113	0R24	0R47
114	0R48	0R32
115	0R70	1R86
116	0R19	1R118
117	0R126	1R28
118	0R122	1R107
119	0R94	1R86
120	0R83	1R98
121	0R87	1R87
122	0R17	1L109
123	0R70	1R101
124	0R96	1R108
125	1L39	1L99
126	0R80	1R78
127	0R120	1R133
128	0R124	1R123
129	0R123	1R128
130	0R44	1R90
131	0L5	0L131
132	0R127	1R59
133	0R11	1R37
134	0R78	1R87
135	0R86	1R134
136	0R12	1R135
137	0L168	1R151
138	0R116	1R151
139	0L164	1L154
140	0R139	1R136
141	0R138	1R140
142	0R19	1R158
143	0R142	1R135
144	0R143	1R142
145	0R141	1R137
146	0R137	1R145
147	0R116	1R134
148	0R12	1R147
149	0R135	1R136
150	0R147	1R78
151	0L149	1R78
152	0R151	1R58
153	0R12	1R152
154	0R150	1R153
155	0R148	1R173
156	0R146	1R155
157	0R126	1R156
158	0R58	1L185
159	0R140	1R58
160	0R158	1R159
161	0R148	1R160
162	0R157	1R161
163	0R78	1R162
164	0R163	1R138
165	0R164	1R160
166	0R86	1R165
167	0R144	1R139
168	0R166	1R167
169	0R12	1R168
170	0R58	1R138
171	0R149	1R159
172	0R170	1R171
173	0L172	1R154
174	0R173	1R153
175	0R174	1R154
176	0R136	1R175
177	0L141	1R139
178	0R177	1R177
179	0R176	1R178
180	0R172	1R179
181	0R169	1R180
182	0R64	1R181
183	0R78	1R142
184	0R183	1R78
185	0R177	1R184
186	0R143	1R185
187	0R186	1R150
188	0R184	1R187
189	0R116	1R188
190	0R183	1R122
191	0R129	0L131
192	0R190	1R191
193	0R189	1R192
194	0R182	1R193
195	0R136	1R194